

Mixture-of-Kittens

MoK 技术排查报告

源码核验、性能口径、相关项目与部署边界

从“单一 megakernel”的表述，
走到可核验的训练执行路径。

研究对象 Blackwell NVL72 上的 MoE 前向与反向实现。

关键发现 核心融合不等于整层一次 launch；固定累加顺序不等于跨 batch 一致；环形缓冲区不等于无限容量。

工程判断 适合进入大域 MoE 训练的候选评估，不宜直接视为任意模型、硬件和 batch 的通用后端。

核验日期 2026 年 9 月 14 日

目标仓库 cursor/mixture-of-kittens

固定快照 22fc95ae6e331a738c4a58a227a8b03cac586e12

快照日期 2026 年 8 月 14 日 (UTC)

基于官方文章、GitHub 源码、基准脚本、相关项目与 issue。包含独立 CPU 规则复算，不包含 MoK CUDA 执行或 NVL72 性能复现。来源链接见文末附录。

报告结论：MoK 是面向 Blackwell NVLink 大域的 MoE 训练执行实现，不是新模型或新路由算法。它将专家计算、跨 GPU 数据搬运和设备端任务调度一起优化。源码能够支撑其核心设计，但“单 kernel”“完全确定性”“最高 2.37 倍”都必须限定范围；它不是任意硬件、任意 MoE、任意 batch 的通用替换件。

核验范围：阅读官方文章、固定版本源代码、基准脚本、相关仓库与 issue；另执行独立 CPU 数学规则检查。本次没有编译 MoK，没有运行 SM100/SM103 CUDA kernel，也没有复现 NVL72 或 512 GPU 性能。文中“作者报告”“社区报告”“源码核实”“分析推导”分别标识证据性质。

0. 核心结论与证据等级

核心问题	排查结论	证据性质
它是什么？	包含前向、反向的 MoE 训练 megakernel 及 Python 调用层，面向共享专家 + 路由专家 + SwiGLU 结构	源码、README
为什么可能更快？	不只加速 GEMM，而是共同安排 token 调度、NVLink 搬运、FFN、缓冲区复用和反向重计算	源码
整个 MoE 层只启动一个 kernel？	不是。主计算通信流水线融合，调度、屏障、复制和最终归约仍在外围	源码
H200 可以直接运行？	当前公开 API 不接受 Hopper；只接受 SM100/SM103	源码
NVL72 就支持 EP72？	不是。当前公开 EP 集合为 1、4、8、16、32、64	源码
训练可以快 2.37 倍？	2.37 倍是作者报告的 MXP8 单层前向最高倍率；其内部端到端训练报告约 1.41 倍	作者报告
完全确定性等于跨框架、跨 batch 完全一致？	不等于；固定求和顺序不保证不同精度、切分或路由输入产生相同位模式	源码 + 分析
能直接替代当前所有 DeepEP / DeepGEMM 路径？	不能这样判断；功能范围不同，且两者已出现 V2 / Mega MoE 等演进	相关项目源码
最值得关注的落地风险？	路由偏斜导致调度容量不足、每层激活保存过大、重计算成本、软件与拓扑耦合	源码、issue

上述结论主要来自 functional 调用链、CUDA scheduler、前向分配代码、基准脚本和相关项目 README，不是只对文章标题作解释。[\[S02\]](#) [\[S03\]](#) [\[S04\]](#) [\[S05\]](#) [\[S10\]](#) [\[S16\]](#) [\[S17\]](#)

1. 它在训练系统中处于什么位置？

1.1 不是新模型，而是 MoE 层的执行后端

MoK 的输入已经包含 `topk_experts` 和 `router_weights`。也就是说，router 选了哪些专家、每个专家分到多大权重，是上游交给它的结果。MoK 负责把这些决定高效执行：把 token 搬到对应专家所在 GPU，执行专家 FFN，再把结果送回并加权求和；反向返回输入、路由权重和专家参数的梯度。[\[S02\]](#) [\[S03\]](#) [\[S12\]](#)

它与几个容易混淆的对象处于不同层级。模型结构决定专家数量、Top-K、共享专家和激活函数；路由器决定 token 选择谁；训练框架管理优化器、并行策略、检查点和训练循环；MoK 则实现其中的 MoE 计算通信路径。它既不是完整的 Megatron，也不是 vLLM/SGLang 一类完整推理服务框架。

源代码接口显式接收共享专家的 `gate/up/down` 三组权重，以及各本地路由专家的对应权重。当前经过核验的标准路径是 SwiGLU FFN，不能仅凭“支持 MoE”推断其直接支持无共享专家、异构专家形状、任意激活函数或带 bias 的所有模型变体。[S03] [S05]

1.2 用数学明确执行内容

对一个 token 的列向量 x ，记第 e 个专家的中间激活为 h_e ：

```
h_e = SiLU(W_gate,e · x) ⊙ (W_up,e · x)
z_e = W_down,e · h_e
y    = z_shared + Σ[e ∈ TopK(x)] g_e · z_e
```

这里的 g_e 是传入 MoK 的路由权重。公开参考实现直接用这些权重作加权求和；上游选择 softmax、sigmoid、归一化或其他路由处理时，必须保证交给 MoK 的语义正确。`d_router_weights` 是对这些输入权重的梯度，不等于 router 投影矩阵的完整梯度，也不代替上游路由函数的链式求导。[S12]

若全局有 E 个路由专家，EP 大小为 R ，并且专家均分，则每个 rank 持有 E/R 个路由专家。源代码采用连续专家编号分段：某个全局专家属于哪个 rank，可以从专家编号与本地专家数计算。将专家映射改为非连续布局时，调度逻辑也必须同步适配。[S04]

1.3 它改变的是关键路径，而非只改变一个算子的速度

传统分段执行中，dispatch、重排、第一组 GEMM、SwiGLU、第二组 GEMM、combine 与归约可能形成多个阶段。即便每个 GEMM 已经很快，整层仍可能被通信、阶段间等待、CPU 提交或路由元数据处理限制。

MoK 的方案是把通信和计算任务放入同一个设备端执行结构，用局部完成计数连接依赖；一部分 SM 搬运 token，另一部分 SM 执行专家计算。源码中不仅存在矩阵乘法，还存在通信任务、共享专家任务、路由专家任务和 CLC 调度句柄。[S05] [S06]

分析推导：将某一串行路径近似写成 $T_{\text{dispatch}} + T_{\text{FFN}} + T_{\text{combine}}$ 时，重叠的目标是减少这几个阶段对总耗时的累加。但真实耗时不会自动变成三者最大值：流水线填充/排空、SM 分配、HBM 争用、负载不均与依赖屏障都可能留下暴露时间。

2. 主执行路径：从路由表到通信计算流水线

2.1 实际调用链

下面是依据 functional 层和 scheduler 重新整理的调用结构；它是源码流程示意，不是实测时间线。[S03] [S04]

```

上游 router: 产生 Top-K 专家编号和路由权重
    ↓
build_schedule
  编号转 int32 → 路由编号 all-gather → GPU 屏障
    → count kernel → pad kernel → schedule kernel
    ↓
functional.forward
  输入复制到 symmetric workspace → GPU 屏障
    → 主 megakernel: pull dispatch + 专家 FFN + push combine
    → GPU 屏障 → fwd_epilogue: 加权归约并合并共享专家
    ↓
  保存 schedule / forward_context, 或按策略丢弃并在反向前重建
    ↓
functional.backward
  复用 schedule, 执行反向通信、输入梯度和参数梯度计算

```

这条调用链解释了“megakernel”的边界：**跨阶段融合是真实存在的，但不能把完整 functional API 描述成只有一次 GPU kernel launch。** `schedule_entrpoint` 本身就启动 count、pad、schedule 三个 kernel；前向外围还存在复制、屏障和 epilogue。[S03] [S04]

2.2 Pull dispatch：由接收端决定自己的连续布局

MoK 的 dispatch 读取本 rank 的调度表，再从远端对称内存读取需要的 token。调度表每项保存两个核心字段：`peer_rank` 和 `peer_token_idx`；后者把原始 token 索引和第几个 Top-K 路由槽编码在一起。调度表的行号则对应本地专家输入的位置。[S04] [S06]

这样做的关键不是“远程读取一定比远程写入快”，而是接收者掌握最终消费布局：自己的某个专家需要哪些 token，哪些数据应放到同一个连续段，何时足够计算一个 minibatch。源码把每个专家的段按 256 行对齐，并在段内交错安排不同来源 rank 的路由，减少长时间只访问少数来源的倾向。[S04]

源代码采用确定的线程遍历顺序，并非简单按原始 token 编号升序排列。阅读或复写 schedule 时，不能把“确定性”偷换成“原始索引有序”；稳定的任意既定顺序都可能满足正确性要求。[S04] [S12]

2.3 Push combine：返回各自的路由槽，再做归约

主 combine 根据同一调度表，把专家输出写回来源 rank 的对应路由槽。`peer_token_idx` 同时编码 token 和 Top-K 位置，因此多个专家对同一 token 的贡献先落到不同槽位，而不是直接以任意到达顺序争抢一个浮点累加位置。最终的加权聚合由外围 `fwd_epilogue` 执行。[S03] [S06]

这既帮助复用调度信息，也为固定归约顺序提供条件。它仍然需要保证写入完成和后续读取可见，不能由此推断整层没有 GPU 同步。可见的 `barrier_all`、TMA completion wait 和局部就绪计数，都属于这个依赖管理的一部分。[S03] [S06]

2.4 一张 schedule 对应同一次路由决定

公开用法在前向构建 schedule，并把它与 forward context 保存给对应反向。概念上，前向 dispatch/combine 与反向的逆向数据流拥有相同的 token—专家关系，因此可复用映射。[S02] [S03]

集成边界：这不代表可以跨任意训练 step、任意层或任意 rollout 重用旧 schedule。只要 token 集合、路由编号、专家分布或相关形状发生变化，就应重新构建或证明缓存仍然有效。尤其不能因为模型参数没变，就假定动态路由必然不变。

3. 两级分块：minibatch 与 macrobatch 不是同一件事

3.1 Minibatch 控制重叠粒度

`minibatch_size` 决定一批路由 token 到齐后，计算侧何时开始消费，以及计算完成后何时通知通信侧。`fwd_num_comm_sms` 与 `bwd_num_comm_sms` 则分别控制前向、反向通信侧资源。默认配置为前向 40、反向 28 个通信 SM，minibatch 为 4096；这些是默认值，不是普遍最优值。[S03]

性能分析：分块太小，流水线更早启动，但屏障频率、任务调度和矩阵乘法尾部开销可能升高；分块太大，矩阵乘法容易吃满，但第一批计算启动更晚，最后一批数据返回更晚。优化目标是最短层耗时，而不是最细粒度、最多通信 SM 或最高单个 GEMM TFLOPS。

共享专家不依赖远端 dispatch，可以在路由专家等待输入时提供本地计算工作。源码在 compute task 空间中显式安排共享专家 gate、up、SwiGLU 和 down 任务；通信 cluster 则走独立的数据搬运路径。[S05]

3.2 Macrobatch 控制可复用的路由激活缓冲区

`macrobatch_size` 控制一段可循环使用的路由激活存储，默认 131072；它必须是 minibatch 的正整数倍。它不是全局优化器 batch size，也不等于每张卡进入 MoE 的原始 token 数。[S03]

当本 rank 实际接收的路由 token 超出一个 macrobatch 时，工作被分成多个段。每段在缓冲区中经历“写入 → 消费 → 输出返回 → 允许覆盖”的生命周期。源码的 `transfer_ready`、`buffer_ready`、`transfer_done` 等计数用于阻止过早覆盖尚未被计算或通信读取的数据。[S06]

3.3 Reversed ring 的实际含义

前向代码确实从 `num_macrobatches - 1` 向 0 处理，并交错执行当前段的 combine 和下一段的 dispatch。这个“反着处理”是对宏批次的执行次序重排，不是反向传播，也不是改变 token 的模型语义。[S05]

分析推导：假设总路由量是 2.3 个缓冲区。按正序处理，最后只留下约 0.3 个缓冲区的有效激活；按反序处理，最后可以留下完整的第 0 段。若反向从该段开始，就能优先利用更多已保存激活，减少必须重建的部分。调度表负责把计算结果放回原 token 位置，因此宏批次执行顺序和模型输出顺序可以解耦。

3.4 CLC 并不等于任意位置立即抢占

前向实现中存在 `clc::handle`、`clc::schedule` 和任务循环。CUDA 官方对 Cluster Launch Control 的描述是：通过取消尚未执行的 block/cluster 并领取其工作，实现设备端 work stealing；高优先级工作可以影响取消是否成功，正在执行的 block 随后完成并释放资源。[S05] [S18]

因此，更准确的理解是改善 persistent grid 与其他 kernel 的调度共存，而不是“正在执行的任意 Tensor Core 指令随时无成本中断”。集成 FSDP 或其他跨机通信时，仍应观察高优先级通信的等待时间与 MoK 吞吐是否相互影响。

4. 精度与确定性：最容易混淆的三组概念

4.1 MXFP8 是计算路径，不代表所有传输都是 FP8

MoK 的 MXFP8 用法是：输入激活仍以 BF16 传入，路由专家权重由调用方提前量化；共享专家保持 BF16。量化参考实现使用 32 元素块、E4M3 数据以及幂次尺度，并提供正常方向和转置方向的量化表示。[S02] [S12]

dispatch 源码从 `activation_bf16_pgl` 指向的远端缓冲区加载 BF16，再在本地暂存过程中生成 MXFP8 数据、scale 与转置表示；combine 同样通过 BF16 输出缓冲区返回结果。[S06]

所以不能根据“MXFP8 模式”就把整个链路的通信量按 BF16 的一半估算。需要分别统计跨 GPU 有效载荷、HBM 读写、量化中间表示和计算输入格式。该固定快照公开的主要精度模式是 BF16 与 MXFP8，不应把文章提及的其他训练精度实践当作此仓库已经实现 NVFP4 的证据。[S02] [S06]

4.2 正常量化与转置量化不一定能靠 transpose 互换

块量化的分组方向会影响每组最大值和尺度。公开参考实现是先对原矩阵量化，同时对转置后的矩阵重新进行块量化，而不是简单转置已经量化的数据及尺度。集成时必须匹配 kernel 预期的布局和 scale 排列。[S12]

训练中参数更新后，也不能无限期复用旧的量化权重。究竟把重新量化放在优化器之后、FSDP 通信前后还是其他位置，需要结合参数布局、缓存版本和训练栈安排；这部分时间应计入端到端比较，而不只看独立 MoE 的计时。

4.3 确定性、数值准确性与 batch invariance 是不同测试

源码对 wgrad 的多 macrobatch 累加有明确处理：第一份贡献用 store，后续贡献使用 add store，并通过 `routed_buffers_done` 串行化相关宏批次，使贡献以固定顺序进入结果。[S07]

不能把这个设计简化成“没有任何原子操作”。scheduler 中存在整数 `atomicAdd`，wgrad 路径也存在加法写回；关键在于涉及结果的浮点累加顺序是否受控，而不是是否出现某个指令名称。[S04] [S07]

测试问题	应当验证的内容	MoK 设计不能自动代替的部分
固定条件的执行确定性	相同输入、权重、路由、配置和环境，多次运行结果是否逐位一致	需要实际 GPU 重复运行和调度扰动测试
相对参考实现的正确性	输出和各梯度的误差是否满足合理标准	确定性不等于误差为零
不同 batch 或分块的不变性	batch 组成、tile、macrobatch 改变后是否得到同一结果	求和分组、量化尺度可能变化
训练与 rollout 一致性	路由、权重版本、精度、logprob 路径是否一致	不由某一个 MoE kernel 独立保证

源码提供了可信的固定顺序机制，但“MoK 在相同配置下可重复”不能推出 BF16 与 MXFP8 相同、H200 与 B300 相同、不同框架逐位相同，也不能推出 MTP 接受率一定上升。这些都是不同的实验命题。

4.4 仓库正确性测试不是普通的逐元素 allclose

`tests/utils.py` 的检查器同时要求最大绝对误差和全局 L1 相对误差满足阈值，并对跨 rank 的统计量做归约。相对误差定义为：

$$L1_relative_error = \sum |reference - actual| / \sum |reference|$$

BF16 的阈值为最大绝对误差 0.5、全局 L1 相对误差 0.01；MXFP8 分别为 1.0、0.1。它不是 `torch.allclose(atol, rtol)` 的逐元素混合容差，也不是逐位确定性测试。[S12]

评估建议：将这些阈值视为该测试集的验收设置，不是任何模型的安全标准。真实训练还应检查梯度范数、余弦相似度、局部极端误差、loss 变化和路由统计，并单独做 bitwise 重复性测试。

4.5 Router 梯度的内积恒等式

对单个专家有 $z = W_down h$ ，令未乘路由系数的下游梯度为 d_y ，则：

$$\partial L / \partial g = d_y^T z = d_y^T W_down h = (W_down^T d_y)^T h$$

这表明计算路由权重梯度不一定要长期保存完整 down 输出，可以利用中间激活与相应 dgrad 的内积。等式本身是线性代数恒等式，前提是明确 dgrad 是否已包含路由系数；不能不加区分地复用已缩放张量。MoK 参考实现返回对应路由权重梯度，文章将其内存优化与 SonicMoE 联系起来。[S12] [S01] [S21]

本次独立 CPU 检查在 100 组 float64 随机样本上验证了该等式，最大绝对差约 5.68×10^{-14} 。这只验证数学变换，不验证低精度 CUDA 实现的误差或并发正确性。

5. 源码揭示的硬边界与容量风险

5.1 运行条件与形状约束

项目	固定快照的公开约束	实际含义
GPU capability	SM100 或 SM103	H100/H200 不在直接支持范围；不能把所有 Blackwell SKU 都视为支持
EP size	1 / 4 / 8 / 16 / 32 / 64	没有 EP2 或 EP72 路径
本地原始 token 数 N	≥ 512 ，且能被 256 整除	小 batch decode 不属于可直接代入的工作负载
hidden size H	正数，且能被 256 整除	模型形状需符合约束
Top-K	1~255，另受动态共享内存限制	255 不是无条件可运行的上限
各 rank 输入形状	N、H、K 相同	变长或 uneven input 需在上游处理
通信 SM 数	正偶数，且少于 GPU 的 SM 总数	前向、反向可独立调参
minibatch / macrobatch	前者按 256 对齐；后者为前者的正整数倍	不是任意连续整数超参数
软件	Python ≥ 3.12 、PyTorch ≥ 2.10 、CUDA ≥ 13.0 ；torch 与 toolkit CUDA 主次版本匹配	安装最低要求与论文基准环境不是同一份配置

硬件、EP 和大部分形状限制来自 `validate_workspace_args` 与 `build_schedule`；软件要求来自 README。底层权重和布局还存在额外校验，以上不是全部错误条件的穷举。[S02] [S03]

5.2 Dropless 的边界：环形激活缓冲区不等于无限调度容量

定义 R 为 EP 大小， N 为本地原始 token 数， K 为 Top-K， α 为 `schedule_capacity_multiplier`。functional 层分配的调度容量为：[S03]

$$C_{\text{schedule}} = N \times K \times \max(2, \text{ceil}(R \times \alpha))$$

对目标 rank 的每个本地专家，实际收到 n_e 条路由，scheduler 按 256 对齐后的总需求为：[S04]

$$T_{\text{padded}} = \sum [e \text{ 为本地专家}] 256 \times \text{ceil}(n_e / 256)$$

源码显式检查：若 $T_{\text{padded}} > C_{\text{schedule}}$ ，执行 GPU trap。因此，固定缓冲区能分段处理激活，不代表任何路由偏斜都能无条件运行，也不是自动触发一个优雅的扩容 fallback。[S04]

一个可构造的例子是：EP=4、N=512、K=2、 $\alpha=0.5$ ，每个 rank 有 4 个专家。如果所有 rank 的 token 都选择 rank 0 上的两个专家，那么 rank 0 需要 4096 个路由槽，而默认公式只分配 2048 个槽。此时 Top-K 没有重复专家，也完全可能满足表面形状约束，但调度容量仍不足。

本次 CPU 规则检查复算了这个例子。它不是报告 GPU 已经崩溃，而是用合法路由输入说明容量条件可以被违反。上线前应根据实际最坏偏斜、专家 padding 和训练初期负载变化设置 α ，并在独立测试中覆盖超限条件。

5.3 增大 EP 不会自动减少平均每卡路由量

分析推导：如果每个 rank 都有 N 个原始 token，每个 token 选择 K 个专家，全局路由总数为 $R \times N \times K$ 。在均衡路由假设下平均分到 R 个 rank，则每个 rank 仍然接收约 $N \times K$ 条路由。

所以，在保持每卡 N 和 K 不变时，单纯把 EP 从 8 加大到 64，不会让平均路由量变为原来的八分之一。变化的是每卡专家数、专家段 padding、网络并发和矩阵任务形状等。只有把全局 token 总量固定并相应降低每卡 N ，才会出现另一种缩放结果。这个区分对理解 issue 中关于“小 EP / 大 macrobatch”的讨论非常重要。[S15]

5.4 CPU-GPU 同步与 GPU-GPU 屏障不能混为一谈

运行时，路由计数留在 GPU 中，不需要为了精确获得本 rank 收到多少 token 而每次把计数读回 CPU。固定容量和设备端分段是实现这一点的关键。但 workspace 初始化会进行跨 rank 形状核对，并出现 `.item()`；清理缓存也会调用 CUDA synchronize。前向外围仍然有 `barrier_all`。[S03]

因此，准确结论是减少或消除热路径上由动态 token 数量导致的 CPU readback，而不是“所有代码不经过 CPU”“整个训练过程没有同步”或“每次调用都完全没有内存分配”。CUDA 前向 host wrapper 仍能看到 `at::empty` 和 `at::zeros` 分配语句。[S05]

6. 显存与重计算：实际接入最需要核算的部分

6.1 至少分开计算三类内存

第一类是 symmetric workspace，例如输入、输出路由槽、输入梯度路由槽、路由权重、all-gather 结果及同步元数据。它可以按适用形状缓存复用。第二类是每次前向返回并需要保存到反向的 `MoKForwardContext`。第三类是计算临时张量与参数、量化副本、优化器和训练框架的其他状态。[S03] [S05]

工作区复用不等于每层激活也已经共享。 同一个模型的多层可以复用某些通信缓冲区，但每层反向需要的前向上下文若仍被保留，生命周期会跨越后续层执行。

6.2 BF16 forward context 的可核算表达式

前向 BF16 分配代码保存四个路由张量：输入 x、gate、up、hidden；其行数都是 macrobatch 大小 M。共享专家保存 gate、up、hidden，行数为原始本地 token 数 N。因此，仅这些被保存的 BF16 张量占用为：[S05]

$$B_context = 2 \times M \times (H + 3I) + 6 \times N \times I \quad \text{bytes}$$

这里不包括权重、量化副本、通信 workspace、输出、调度表、反向暂存、allocator 保留量或其他层激活。该表达式针对已核验的 BF16 保存路径；不要直接套用到 MXFP8。

例如 N=16384、H=6144、I=2048、M=131072 时，结果是 **3264 MiB/层**。若 61 层同时保存这些上下文，仅这一项约为 194.4 GiB。这个数字说明 default macrobatch 不是“越大越好”；它可以显著改善流水线连续性，但也可能挤占整个训练预算。[S05] [S15]

6.3 Issue #14 的反馈与当前修复状态

一位实际接入者在 2026 年 8 月 12 日报告了 EP8、GB300、BF16、大 token 数下的内存/速度取舍，测试基于旧提交 6438bf4。下表摘录其给定配置的测量，不是本报告复测结果，也不是任何硬件上都成立的性能规律。[S15]

Macrobatches	保存 context / 层	前向 ms	反向 ms	前向 + 反向 ms
262144	6336 MiB	9.19	17.52	26.71
131072	3264 MiB	9.51	18.99	28.50
65536	1728 MiB	9.66	24.58	34.24
32768	960 MiB	9.73	28.24	37.97
16384	576 MiB	9.72	32.11	41.83

缩小 M 确实减少了保存量，但该用户看到反向显著变慢。可能成本既包括前向激活重建，也包括更多段间依赖、流水线重新填充、wgrad 的更短归约段和重复写回。源码中固定 wgrad 累加顺序的设计也说明：确定性、并发程度和存储开销需要一起权衡。[S07] [S15]

维护者在 **2026 年 8 月 14 日** 加入 `recompute_forward_context`，本报告固定的主分支快照已经包含这一接口。因此，不能再把“完全没有手动上下文重建接口”列作当前缺陷。[S14] [S15]

但 issue 关闭也不表示所有 shape 的速度/显存矛盾已经消失。新接口允许前向丢弃 context，在反向前重建必要部分；是否更快、更省内存，取决于它与外层 activation checkpointing 的协调。如果外层完整重算 MoE，内层又发生 ring replay，就可能付出重复计算。

7. 性能数字应如何解读？

7.1 作者公开结果：单层和整训分开看

层级	模式	作者报告的结果	不能外推的内容
单 MoE 层前向	MXFP8	相对测试中最快基线，最高 2.37×	不是整模型或整训 2.37×
单 MoE 层反向	MXFP8	最高 1.78×	不能与前向最高倍率相乘
单 MoE 层前向	BF16	最高 1.92×	不代表所有 shape 都有该收益
单 MoE 层反向	BF16	最高 1.58×	不代表相同 shape 同时达到其他最高倍率
内部端到端训练	512 GPU	760.9 → 1070.2 tokens/s/GPU，约 1.41×	不能仅靠开源层基准完整复现内部训练栈

四个单层倍率来自仓库 README。文章将单层条件限定为一个 GB300 NVL72 机架中的 EP64、每卡路由前 2048 tokens；512 GPU 数字来自多个 GB300 NVL72 机架上的内部生产训练比较。[S02] [S01]

本报告复算： $1070.2 / 760.9 = 1.40649$ ，即吞吐提高约 40.65%；若工作量固定且其他条件完全相同，所需时间下降约 28.90%。“吞吐提升 41%”不是“训练时间下降 41%”。

7.2 脚本中包含什么、不包含什么？

公开 `bench_mok.py` 在前向函数内部调用 `build_schedule`，所以前向计时不是只测主 megakernel。初始化 symmetric workspace 和权重 MXFP8 量化则发生在 benchmark 对象构造阶段，不在每次前向计时里。反向利用对应前向保存的 schedule/context，计时器从前向完成后才开始覆盖反向调用。[S10] [S11]

输入由固定种子的随机分布生成；router logits 来自随机高斯采样，然后取 Top-K 和 softmax。它适合比较相同合成条件下的实现，不等价于真实训练中所有专家偏斜、跨 step 相关性或极端 rank 热点。[S12]

计时公共工具使用 500 次 warmup、100 次测量，用 CUDA events 记录每个 rank 的耗时，再取每次测量的 rank 最大值，最后取中位数。这比只看 rank 0 更能反映同步训练的慢 rank，但它不是 P95/P99 尾延迟报告。工具没有把“完整优化器 step、数据加载、checkpoint、权重重新量化”等自动纳入单层指标。[S11]

有效 GEMM 计算量按 $6 \times N \times (1+K) \times H \times I$ 计算前向 FLOPs，反向按两倍估计。这是算法级有效吞吐口径，不包含所有 padding 和 replay 实际执行的额外计算，因此不能直接等同于硬件峰值利用率。[S11]

7.3 基线并非只有朴素 PyTorch

仓库含 NCCL+PyTorch、DeepEP+PyTorch、DeepEP+Transformer Engine，以及 HybridEP+Megatron 基准。HybridEP 路径启用 TE 的 fused grouped MLP、共享专家 overlap 等设置，并检查算子融合实现是否激活。因此，不应在未阅读脚本时断言其“只找最慢 baseline 比”。[S13]

同时必须保留具体版本。源码头部记录的环境包括：

组件	基准脚本记录
DeepEP（TE 路径）	2.1.0 + dd758caf
DeepEP hybrid-ep	f725d29
Megatron Core	0.19.0 + 602fad0
Transformer Engine	2.17.0 + 2e559f06
PyTorch	2.13.0 + cu130
NCCL / cuBLAS / CUDA	2.30.7 / 13.6.0.2 / 13.0

这些版本来自基准文件头部，不是本报告环境的安装状态。特别是 DeepEP 已明确标为 **2.1.0**，不能把该对比错误描述为“只测了 DeepEP V1”。反过来，这份版本记录也不构成对未来 main 分支、其他调优配置或所有 workload 持续领先的证明。[S13]

7.4 用 Amdahl 定律校验收益预期

若原始整训时间中 MoE 占比为 f，MoE 平均加速 s 倍，且其他部分不变，则：

$$S_{total} = 1 / ((1 - f) + f / s)$$

例如 f=60%、s=1.8，则整训加速约 **1.36×**。这是一个计算示例，不是 MoK 的实测结果。真实训练中重叠关系、FSDP 通信、显存压力和 CPU 调度也会改变，不能把这个简单模型当作精确预测。

尚缺的公开证据：本次未取得内部 512 GPU 完整训练配置、原始 traces、全部逐 shape 测量数据与匹配的长期质量实验，因此不报告“独立复现 1.41 倍”或“各种训练任务均无数值风险”。

8. 与 GitHub 相关项目的关系

8.1 先区分依赖、思路关联与替代方案

项目	与 MoK 的关系	核验到的范围	阅读时的重点
ThunderKittens	直接源码依赖	.gitmodules 明确引用，CUDA 文件包含 kittens 头文件	底层 tile / TMA / kernel 编程抽象
ParallelKittens	相关多 GPU kernel 研究	讨论多 GPU 计算通信重叠和编程简化	不应凭名字断言整个 MoK 基于其框架
DeepEP	通信方案与基线	当前 README 有 V2 / ElasticBuffer / NCCL Gin；MoK 基准固定到具体版本	传输模式、SM 占用、规模与通信语义
DeepGEMM	计算库及融合 MoE 对照	当前不只有 GEMM，也有 Mega MoE	是否匹配前向/反向、精度和共享专家语义

项目	与 MoK 的关系	核验到的范围	阅读时的重点
HybridEP + Megatron + TE	公开集成栈基线	MoK 仓库含可读 benchmark 适配代码	应比较整层，而不只是孤立 GEMM
COMET / SonicMoE	相关优化研究	分别涉及计算通信重叠、IO/tile 感知优化	用于理解设计谱系，不混用不同条件的性能倍率

直接依赖关系由 `.gitmodules` 和 `include` 核实；其余分类依据相应论文与官方项目文档。没有证据表明 MoK 直接依赖 DeepEP 或 DeepGEMM 来实现自己的主 megakernel。[S08] [S16] [S17] [S19] [S20] [S21]

8.2 ThunderKittens 与 “没有框架” 不是矛盾

MoK CUDA 源码使用 `kittens.cuh`、TMA、tile、warpgroup 等抽象。它不是把所有底层工具都重新实现一遍。所谓更直接地构造 megakernel，与仍然使用底层 GPU 编程库可以同时成立。技术阅读时应分别看“底层 primitive 库”和“上层通用 megakernel 调度框架”，不要把它们统称为一个 framework 后作二选一判断。[S08] [S04] [S05]

ParallelKittens 的官方论文关注多 GPU AI kernel 的系统化简化与计算通信重叠；它有助于理解这一类设计，但本次核验到的 MoK git submodule 是 ThunderKittens，不能仅从作者或命名关联推断所有代码继承关系。[S19] [S08]

8.3 DeepEP：必须按当前版本讨论

当前 DeepEP README 将 V2 描述为采用 NCCL Gin、统一高吞吐和低延迟 EP 的 `ElasticBuffer` 接口，并扩展到更大的 scale-up/scale-out 范围；还包含实验性 PP、CP、Engram 等能力。因此，不能继续笼统地把 DeepEP 描述成“始终只有某一种旧式粗粒度 push 实现”。[S16]

MoK 与它最核心的差别，是本次研究对象覆盖了具体专家 FFN 的前向/反向计算和通信协同，而不是只提供通信组件。比较时，应把 DeepEP 与相应计算后端、排列/反排列、激活和归约一起组合成完整 MoE 层。

同样，不能因为 DeepEP 有新 README 就否认 MoK 对现有基线的测试：MoK 的脚本已经注明 DeepEP 2.1.0。严谨做法是保留被测 commit，再在目标环境重新比较，而不是靠“新版本一定更快”或“文章一定已经过时”下结论。[S13] [S16]

8.4 DeepGEMM：已经不能只叫它 “GEMM 库”

当前 DeepGEMM 官方 README 明确提供 Mega MoE：融合 EP dispatch、两层线性投影、SwiGLU 和 EP combine，利用 NVLink 计算通信重叠，并依赖 symmetric memory；公开示例支持 FP8×FP4 或 FP8×FP8 路由专家权重路径。其 NEWS 还记录了 2026 年 9 月 10 日的 MoE 等优化更新。[S17]

因此，“MoK 是完整融合、DeepGEMM 永远只做乘法”的对比已经不准确。Mega MoE 应进入候选对照集合。

不过，本次核验的 Mega MoE README 示例展示的是前向入口，不能据此确认它与 MoK 具有相同的完整反向、共享专家布局或确定性契约。DeepGEMM 有独立 wgrad GEMM 算子，也不等于已经核实存在语义对等的 fused Mega MoE backward。需要比较的，是满足相同要求的具体 API 和实现，而不是仓库名字。[S17]

8.5 COMET 与 SonicMoE：不是性能数字可以直接拼接

COMET 的研究主题就是细粒度计算通信重叠，说明“重叠通信与 MoE 计算”不是 MoK 首次提出的方向。SonicMoE 关注 IO 与 tile 感知优化，可以帮助理解保存哪些激活以及如何减少数据搬运。这些工作与 MoK 有技术关联，但本报告未做同硬件、同形状、同精度的交叉实测，不给出伪造的排行榜。[S20] [S21]

9. 对训练、RL 与 H→B 迁移意味着什么？

9.1 适合优先评估的场景

工程判断：拥有 SM100/SM103 与合适 NVLink 大域，模型接近已实现的共享专家 + routed SwiGLU，且 profiler 表明 MoE 通信和阶段等待明显暴露的训练任务，最值得优先评估。模型规模大、EP 大并不是充分条件；还需要实际瓶颈位于 MoK 能改变的路径。

对于预训练、SFT 或 RL 的训练更新阶段，只要底层 MoE 执行语义相同，核心评估方法类似。损失函数不同，不会自动让某个 kernel 适用或不适用；需要检查真实 token 形状、路由偏斜、并行组合和重计算设置。

9.2 不应优先承诺收益的场景

Hopper/H200 不能按公开 API 直接运行，低于 512 或不满足对齐的每卡 token 数也不能直接输入。纯小 batch 自回归 decode、更复杂的异构专家结构、跨多个 NVLink 域的直接 EP 传输，都不能由当前源码的支持范围自动覆盖。[S03] [S06]

512 GPU 的端到端报告也不能解释为“一个 MoK EP group 横跨全部 512 张卡直接做 RDMA”。主接口限制最大 EP64，且所核验数据平面操作是对 peer symmetric buffer 的访问。多机架训练可以包含多个 EP group，以及由训练栈处理的其他并行通信；具体组合需要完整训练配置才能确认。[S01] [S03]

9.3 对 Router Replay 和 rollout 一致性的关系

分析判断：MoK 的设备端计算确定性能够减少一类“同输入重复执行却出现差异”的来源，便于归因实验。但 Router Replay 固定的是专家选择；batch invariance 关注不同 batch 切分下的结果；训练/rollout 一致性还涉及模型权重版本、量化、其他层、采样和 logprob 计算。它们不能互相替代。

从 Hopper 迁移到 Blackwell 时，应该分两层验收：先验证模型/路由/梯度语义和允许的数值误差，再验证在目标 Blackwell 环境中重复执行是否稳定。把“H200 与 B300 必须逐位一致”作为所有优化的必要条件，通常不是对不同执行路径的合理默认要求；反过来，也不能以“都是浮点误差”为由忽略可能影响训练的系统性漂移。

10. 可执行的复测与接入方案

10.1 第一步：固定源码、拓扑与软件

保留 MoK commit、ThunderKittens submodule commit、GPU 型号/数量、driver、CUDA toolkit、PyTorch 构建、相关通信库版本和全部环境变量。对 EP group 的每对 GPU 核实所需 peer/symmetric memory 能力，而不是仅检查 `nvidia-smi` 看起来有很多卡。

以下命令是依据公开安装和测试入口整理的示例，**本次没有执行**。它们假定已经安装兼容 PyTorch、CUDA、pytest，且在正确的 GPU 容器或节点环境中运行。[S02]

```
git clone --recursive https://github.com/cursor/mixture-of-kittens.git
cd mixture-of-kittens
git checkout 22fc95ae6e331a738c4a58a227a8b03cac586e12
git submodule update --init --recursive

# SM103: 例如相应 GB300 平台; SM100 平台改成 SM100。
MOK_ARCH=SM103 python -m pip install . --no-build-isolation
python -c 'import mok; print("MoK import OK")'

# 单节点 4 GPU 示例: 用于功能检查, 不代表 EP64 性能。
torchrun --standalone --nproc-per-node=4 \
-m pytest -s tests/test_functional_e2e.py
```

安装前还应确认 PyTorch 的 CUDA 主次版本与 toolkit 一致。复现文章数据时, 应优先匹配基准脚本记录的具体依赖版本, 而不是只满足 README 的最低版本条件。

10.2 第二步: BF16 对齐正确性, 然后再上 MXFP8

对输出、输入梯度、路由权重梯度和六组专家参数梯度逐项检查; 让共享专家路径与路由专家路径可以分别定位误差。加入空专家、单专家热点、单 rank 热点、256 对齐边缘、macrobatch 边界和多个宏批次的测试。不要只用随机均衡分布通过一次就进入大规模训练。

建议采用以下验收矩阵; 它是本报告的测试设计, 不是仓库已覆盖范围的声明。

维度	建议覆盖	要记录的信号
数值	BF16、MXFP8; 真实尺度与极端值	最大绝对误差、L1/L2 相对误差、梯度余弦、非有限数
路由	近均衡、真实 trace、专家热点、rank 热点	最大/平均接收量、padding 比例、容量余量
分块	单 macro、多 macro、尾部 partial macro	输出/梯度、重计算次数、峰值显存、段间等待
调度扰动	相同输入重复运行, 并加入竞争任务	bitwise hash、慢 rank、等待时间
检查点	保存 context、手动重建、外层 AC	重算次数、持久激活量、整步耗时
并行组合	与 FSDP 等通信共存	通信暴露量、吞吐、OOM、trace 中的空洞

10.3 第三步: 独立调优, 不复制别人的最优点

建议先固定精度、路由和形状, 分别扫描前向/反向通信 SM 数, 再扫描 minibatch; 随后在显存预算内调整 macrobatch, 并记录是否触发额外 replay。README 给出的通信 SM 与 minibatch 推荐范围可以作为起点, 但不能代替目标 workload 的测量。[S02]

基准脚本提供形状和通信 SM 环境变量, 例如:

```
# 已具备单节点 4 张兼容 GPU 的示例。
# 目标 EP64 应通过实际集群启动器启动 64 个合适的 EP ranks。
NUM_LOCAL_TOKENS=2048 \
HIDDEN_DIM=7168 INTERMEDIATE_DIM=3072 \
NUM_EXPERTS=384 TOPK=6 \
BF16_FWD_COMM_SMS=24 BF16_BWD_COMM_SMS=28 \
MXFP8_FWD_COMM_SMS=36 MXFP8_BWD_COMM_SMS=36 \
MINIBATCH_SIZE=4096 MACROBATCH_SIZE=131072 \
torchrun --standalone --nproc-per-node=4 -m benchmarks.bench_mok
```

以上形状和默认通信参数来自脚本，命令没有承诺在特定小显存配置下不会 OOM，也不代表 4 GPU 结果可以复现文章的 EP64 倍率。[S10]

10.4 第四步：把隐藏成本重新放回训练循环

真正有决策价值的最终对比，应覆盖权重重新量化、输入复制、route schedule、主 kernel、epilogue、激活重建、参数通信和优化器更新。对同样训练样本与质量要求，比较整步时间、有效 tokens/s、显存峰值和训练稳定性。

对于多流或并发层调用，还要审查缓存 workspace 的所有权与生命周期。复用相同 scratch buffer 的调用若同时执行，可能需要额外的串行化或独立工作区；仅仅看到 `get_workspace` 缓存命中，不等于并发使用安全。本报告将此列为集成检查项，而不是宣称已经复现了一个并发 bug。[S03]

最终 baseline 至少应包含团队已有实现、与目标精度相匹配的 DeepEP + 最佳计算后端、以及功能对等时的其他 fused MoE 方案。缺少反向或共享专家等必要能力的候选，可以作局部前向参考，但不能混入“完整训练层”排名。

11. 本次实际完成的独立验证与尚未完成项

11.1 已执行：CPU 数学规则检查

本次另写了一个独立 NumPy 检查程序，不导入、不复制运行 MoK 的 CUDA 扩展。程序按已公开的调度规则构造路由，验证槽位唯一、专家归属正确、路由没有遗漏，并与独立 round-robin 参考排列比较。

检查项	本次结果	结论边界
36 组路由调度场景	全部通过，合计检查 198400 条路由	支撑规则层面的映射正确性，不证明 CUDA 并发安全
热点容量反例	需求 4096 槽，容量 2048 槽	说明默认容量可被合法偏斜路由超过
100 组 router 梯度内积恒等式	最大 float64 绝对差约 5.68×10^{-14}	验证代数等价，不验证 BF16/MXFP8 实现误差
显存与性能复算	3264 MiB; $1.40649 \times$; 时间下降 28.90%	算术核对，不是新的硬件性能测量

11.2 未执行或未取得

没有在 SM100/SM103 上运行官方单测、压力测试、CUDA race 检查或性能基准；没有完整复现 Cursor 的内部生产训练配置；没有证明所有支持形状都 bitwise deterministic；没有在相同条件下跑 DeepEP、DeepGEMM Mega MoE 与 MoK 的新一轮横向对比。

因此，本报告足以回答“它是什么、设计是否有实现、有哪些明显边界、该如何评估”，但**不把静态核验包装成独立性能背书或生产准入结论**。

12. 最终判断

MoK 最有价值的地方，是把 MoE 的优化单位从单个 GEMM 或通信算子提升到完整的数据依赖路径：目的端布局驱动的拉取、复用路由映射、可调通信计算粒度、反向保存/重建、确定的累加顺序，以及与 Blackwell 调度机制的结合。相关实现可以在代码中逐项找到，而不只是宣传语。[\[S03\]](#) [\[S04\]](#) [\[S05\]](#) [\[S06\]](#) [\[S07\]](#)

建议把它列为“Blackwell 大域 MoE 训练的高优先级候选”，而不是“立刻全量替换的通用后端”。第一轮试验应优先回答两个问题：目标层中通信等待是否真的暴露；在完整训练的显存与重计算预算下，MoK 能否继续保持收益。

如果这两个问题都成立，再投入到框架接入、真实路由 trace、FP8 训练质量与多机架并行联调；如果并不成立，继续优化现有 DeepEP/TE/DeepGEMM 路径也可能更划算。这个结论来自瓶颈分析和支持范围，而不是项目名气或 GitHub 星数。

附录 A：源码阅读顺序

顺序	文件 / 入口	应回答的问题
1	README.md	支持范围、安装、公开调用方式是什么？
2	mok/functional.py	调度、复制、屏障、主 kernel、context 的真实边界在哪里？
3	csrc/scheduler.cuh	路由如何落到连续专家段？padding 和超容量如何处理？
4	csrc/megakernel/dispatch_combine.cuh	pull/push 如何实现？传输精度、完成等待与覆盖保护是什么？
5	csrc/megakernel/forward.cuh	通信/计算任务如何组织？反序 macro 和 CLC 如何使用？
6	csrc/megakernel/grouped_gemm.cuh	GEMM epilogue、量化与 wgrad 累加如何保持顺序？
7	csrc/megakernel/backward.cuh	按目标集成继续审查反向依赖、重建与参数梯度生命周期
8	benchmarks/bench_mok.py、benchmarks/ utils.py	到底计了什么时间，是否包含预处理和初始化？
9	tests/utils.py、tests/ test_functional_e2e.py	参考语义和误差标准是什么？还缺哪些真实分布测试？

附录 B：来源与可访问链接

除特别说明外，MoK 源码链接固定到同一 commit，便于复核。相关项目的 main 分支内容以 2026 年 9 月 14 日本次读取为准；其后变化不在本报告结论范围内。

[S01] **Cursor 官方技术文章**，发布于 2026-08-04。主要用于作者性能结果、测试部署条件与设计动机。

<https://cursor.com/cn/blog/mixture-of-kittens>

[S02] **MoK README**：项目定义、精度、依赖要求、安装、配置和前向/反向示例。

<https://github.com/cursor/mixture-of-kittens/blob/22fc95ae6e331a738c4a58a227a8b03cac586e12/README.md>

[S03] **MoK functional 层**：MoKConfig、workspace、验证、build_schedule、forward、backward。

<https://github.com/cursor/mixture-of-kittens/blob/22fc95ae6e331a738c4a58a227a8b03cac586e12/mok/functional.py>

[S04] **CUDA scheduler**：计数、256-padding、源 rank 交错、容量 trap。

<https://github.com/cursor/mixture-of-kittens/blob/22fc95ae6e331a738c4a58a227a8b03cac586e12/csrc/scheduler.cuh>

[S05] **前向 megakernel 与 host 分配**：任务分类、CLC、反序宏批次、BF16 context 分配。

<https://github.com/cursor/mixture-of-kittens/blob/22fc95ae6e331a738c4a58a227a8b03cac586e12/csrc/megakernel/forward.cuh>

[S06] **Dispatch / combine**：远端 BF16 TMA 访问、接收端量化、缓冲区复用同步。

https://github.com/cursor/mixture-of-kittens/blob/22fc95ae6e331a738c4a58a227a8b03cac586e12/csrc/megakernel/dispatch_combine.cuh

[S07] **Grouped GEMM**：wgrad 固定顺序写回、epilogue 与量化。

https://github.com/cursor/mixture-of-kittens/blob/22fc95ae6e331a738c4a58a227a8b03cac586e12/csrc/megakernel/grouped_gemm.cuh

[S08] **直接依赖记录 .gitmodules**；ThunderKittens 主仓库。

<https://github.com/cursor/mixture-of-kittens/blob/22fc95ae6e331a738c4a58a227a8b03cac586e12/.gitmodules>

<https://github.com/HazyResearch/ThunderKittens>

[S09] **官方端到端功能测试**：输出、梯度和前向 context 重建测试。

https://github.com/cursor/mixture-of-kittens/blob/22fc95ae6e331a738c4a58a227a8b03cac586e12/tests/test_functional_e2e.py

[S10] **MoK benchmark**：初始化预量化、计时前向内构建 schedule、参数变量。

https://github.com/cursor/mixture-of-kittens/blob/22fc95ae6e331a738c4a58a227a8b03cac586e12/benchmarks/bench_mok.py

[S11] **Benchmark 公共工具**：warmup、计时、rank 最大值中位数与 FLOPs 口径。

<https://github.com/cursor/mixture-of-kittens/blob/22fc95ae6e331a738c4a58a227a8b03cac586e12/benchmarks/utils.py>

[S12] **测试参考实现**：随机路由、量化、schedule 参考、MoE 梯度及误差定义。

<https://github.com/cursor/mixture-of-kittens/blob/22fc95ae6e331a738c4a58a227a8b03cac586e12/tests/utils.py>

[S13] **优化基线与精确版本**：HybridEP+Megatron，以及 DeepEP+TE。

https://github.com/cursor/mixture-of-kittens/blob/22fc95ae6e331a738c4a58a227a8b03cac586e12/benchmarks/bench_hybrid_ep_megatron.py

https://github.com/cursor/mixture-of-kittens/blob/22fc95ae6e331a738c4a58a227a8b03cac586e12/benchmarks/bench_deep_ep_te.py

[S14] **固定快照提交**: 新增 forward context 重建 kernel 与 API, 2026-08-14。

<https://github.com/cursor/mixture-of-kittens/commit/22fc95ae6e331a738c4a58a227a8b03cac586e12>

[S15] **Issue #14 及维护者回复**: EP8 激活内存/重计算反馈、旧版本数据与后续接口落地。

<https://github.com/cursor/mixture-of-kittens/issues/14>

[S16] **DeepEP 官方 README**: V2、ElasticBuffer、NCCL Gin 和相关通信范围。

<https://github.com/deepseek-ai/DeepEP/blob/main/README.md>

[S17] **DeepGEMM 官方 README**: Mega MoE、相关 API、近期更新及 wgrad GEMM。

<https://github.com/deepseek-ai/DeepGEMM/blob/main/README.md>

[S18] **NVIDIA CUDA 文档: Cluster Launch Control**。

<https://docs.nvidia.com/cuda/cuda-programming-guide/04-special-topics/cluster-launch-control.html>

[S19] **ParallelKittens 官方论文页面**。

<https://arxiv.org/abs/2511.13940>

[S20] **COMET 官方论文页面**。

<https://arxiv.org/abs/2502.19811>

[S21] **SonicMoE 官方论文页面**。

<https://arxiv.org/abs/2512.14080>